Enterprise Test Strategy: UPI & IMPS Payment Modules

1. Introduction & Context

This strategy document serves as the master plan for validating the **Payment (Fund Transfer)** module. As an expertise tester in the **BFSI domain**, the focus is on achieving a **Zero-Failure mandate** for Happy Path scenarios while ensuring stability through modern automation.

2. Strategic Framework (ICEPOT/ICETOP)

Following the **ICETOP Framework**, this strategy ensures clear instructions and expert-level personas:

- **I - Instructions**: Generate 10 total test cases (5 UPI, 5 IMPS) focusing on P1/P2 Happy Path scenarios.
- **C - Context**: Act as a Banking Domain Assistant expertise in functional testing.
- **E - Examples**: Utilize one-shot or few-shot patterns for JSON formatting.
- **P - Persona**: Managed by a **Test Automation Architect** and reviewed by a **Domain Lead**.
- **O - Output**: Display results in **Table and JSON formats** for standard industry reporting.
- **T - Tone**: Strictly **Professional**.

3. Technical Strategy & Tools

Component	**Specification**
Automation Tool	**Playwright** (Enterprise-grade following industry standards).
Environment	UAT using **Mocked APIs** for payment gateways to ensure deterministic results.
Constraint	**[STRICT]** Use explicit waits (Playwright auto-waits); **[MANDATORY]** No Thread.sleep.
Success Criteria	**Zero-Failure** for [P0] Happy Path; execution of critical negative scenarios.

4. Execution Scenarios (Summary)

- **UPI Transfer**: Verify successful fund transfer with valid UPI ID, balance reflection, and transaction history.

- **IMPS Transfer**: Validate immediate payment settlement and reference number generation.
- **Happy Path [CRITICAL]**: E2E scenarios only for successful transactions.

5. Team Lead Review Comments

As a Domain Expertise Team Lead, I have reviewed the generated strategy:

ID	Review Category	Comments / Suggestions	Status
REV_01	**Tooling**	Transitioning to Playwright from Selenium supports better handling of async banking UI elements.	**Approved**
REV_02	**Stability**	Use of Mocked APIs is critical to avoid "flaky" tests that lead to blame in production.	**Approved**
REV_03	**Priority**	[P0] and [P1] functional categories are strictly followed as per requirements.	**Approved**

6. TL;DR (Executive Summary)

This strategy implements a high-reliability testing framework for **UPI and IMPS** using **Playwright** and **Mocked APIs**. By adhering to the **ICETOP** model and **Zero-Failure** mandates, we ensure that production-critical steps are guarded against regression.

Playwright Automation Framework Structure

This structure is organized to ensure modularity, scalability, and ease of maintenance for UPI and IMPS modules.

Plaintext

```
playwright-bfsi-automation/
├── .github/workflows/        # CI/CD pipeline configurations (Zero-Failure Gates)
├── data/                     # Centralized Mock Utility [MANDATORY]
│   ├── mocks/                # JSON response payloads for UPI/IMPS
│   │   ├── upi-success.json
│   │   ├── imps-failure.json
```

```
│   └── constants.ts          # Fixed test data and VPA IDs
├── pages/                    # Page Object Model (POM) [STRICT]
│   ├── base.page.ts          # Common actions (waits, clicks, assertions)
│   ├── login.page.ts         # Login & MFA handling [cite: 33]
│   ├── payment.page.ts       # UPI/IMPS transfer UI elements
│   └── history.page.ts       # Transaction status validation
├── tests/                    # Test Suites
│   ├── upi/
│   │   ├── upi-happy-path.spec.ts [CRITICAL]
│   │   └── upi-negative.spec.ts
│   └── imps/
│       ├── imps-happy-path.spec.ts [CRITICAL]
│       └── imps-negative.spec.ts
├── utils/                    # Shared Utilities
│   ├── api-mocker.ts         # Centralized logic for route.fulfill()
│   └── logger.ts             # Professional logging for review evidence
├── .env                      # Environment variables (Credentials/URLs)
├── playwright.config.ts      # Global Config (HTML Reports, Projects, Retries)
└── package.json              # Dependencies and Scripts
```

Framework Architectural Components

1. Centralized Mocking Layer

To meet the **Zero-Failure** requirement, we use a dedicated utility to intercept network calls.

- **Logic:** Instead of hard-coding responses in tests, the api-mocker.ts utility reads from the data/mocks/ directory.

- **Benefit:** Ensures tests remain deterministic and are not affected by third-party payment gateway downtime.

2. POM with Strict Constraints

- **Explicit Waits:** Every page object uses Playwright's auto-waiting features; **[MANDATORY]** no Thread. Sleep or hard-coded timeouts are allowed.

- **Stable Locators:** Uses user-facing attributes (e.g., getByRole, getByLabel) to ensure tests survive UI updates.

3. Global Configuration (playwright.config.ts)

- **Reporting:** Configured for the **Standard Playwright HTML Report**.

- **Artifacts:** Screenshots and trace files are captured for every failed step to provide evidence for the Team Lead review.

- **Projects:** Defines execution profiles for "Mocked" vs "Live" environments to allow for seamless switching.

4. Zero-Failure Validation Logic

- **Business Assertions:** Tests do not just check for page loads; they validate specific transaction reference numbers and balance updates.
- **Critical Path Failures:** Any failure in a [P0] step (Happy Path) is configured to immediately stop the pipeline and alert the team.

Team Lead Design Review

Review Category	Architect's Note	Status
Maintainability	Centralized mocks and POM structure minimize maintenance effort when the UI or API changes.	✅ Approved

Review Category	Architect's Note	Status
Reliability	Native Playwright TypeScript support reduces "flakiness" associated with legacy Selenium wrappers.	✅ Approved
Accountability	The HTML report and Trace Viewer provide the "clear evidence" required to defend decisions in review meetings.	✅ Approved

1. Structural Analysis: Automation Framework

The designed framework structure adheres to the principles of modularity and strict BFSI security standards.

Component	Architecture Assessment	Alignment
POM Layer	**Separating selectors from logic ensures that UI changes in the banking portal don't break the entire suite.**	✅ **High**
Centralized Mocking	**Essential for UPI/IMPS. It eliminates dependency on the NPCI gateway or third-party bank uptime.**	✅ **Optimal**
Playwright Config	**Use of "Projects" for environment switching allows the same code to run in "Mocked" and "Integration" modes.**	✅ **Professional**

---

2. Strategic Alignment Review (ICETOP)

I have audited the strategy against your specific constraints:

- **Instruction [I]: The transition from Selenium to Playwright was handled successfully, maintaining the explicit wait logic while leveraging Playwright's native speed.**
- **Context [C]: The BFSI domain knowledge is reflected in the focus on transaction reference numbers and account balance validation.**
- **Persona [P]: The "Architect" persona enforced a dry, scalable structure, while the "Team Lead" persona ensured reporting visibility.**

- Tone [T]: The output maintains a professional, industry-standard technical tone.

---

3. Risk & "Zero-Failure" Assessment

Risk Factor	Review Mitigation	Rating
Flakiness	By enforcing [MANDATORY] auto-waits and [STRICT] no Thread.sleep, we've minimized the primary cause of BFSI test failure.	🟢 Low Risk
False Positives	Centralized JSON mocks prevent tests from passing just because the "UI loaded"—they must see the specific mock data.	🟢 Low Risk
Production Blame	The use of HTML Trace Viewers provides forensic evidence for every step, protecting the engineer from "it worked on my machine" disputes.	🟢 Low Risk

---

4. Final Review Comments (Team Lead Persona)

Summary: The framework is technically sound and ready for implementation.

- Strengths: The use of TypeScript provides the "Strictly Typed" safety needed for financial calculations. The centralized mock utility is the standout feature that will ensure the Zero-Failure mandate.
- Critical Recommendation: Ensure that the api-mocker.ts utility is integrated into the base.page.ts so that every test session automatically boots with the correct interception rules.
- Verification: The [P0] Happy Path scenarios are properly gated. If these fail, the CI/CD pipeline will stop, fulfilling the safety requirement.

Executive Summary

The generated strategy and framework design are fully approved. They represent a modern, enterprise-standard approach that replaces legacy Selenium patterns with a high-performance, deterministic Playwright TypeScript solution. The focus

on Mocked APIs and POM ensures that the "Zero-Failure" goal is technically achievable.

Status: READY FOR SCRIPTING